Coordinate Sparse Matrix - Vector Multiplication with MPI

Carmen Park

December 6, 2025

1 Introduction

Sparse matrix - vector multiplication is a fundamental operation in scientific computing. It has a multitude of applications and has been an area of focus for some of our high performance computing projects. For the final assignment in this course, we are implementing a naive coordinate SpMV using message passing interfaces, or MPI. MPI is a standardized communication protocol and library specification that can be used in parallel computing environments.

Some fundamental aspects of MPI are how it processes and executes the same program with different processes. This is known as the single program multiple data model. While in this project we only had time for a naive coordinate implementation, there is scaffolding to support a two-dimensional coordinate MPI implementation.

2 Implementation Approach and Methodology

To complete the given function, we were assigned a set of tasks:

- **Step 1: Calculate work distribution**
  - Implement work distribution logic on rank 0
  - Calculate `nnz_per_rank` properly for all ranks
- **Step 2: Broadcast setup data**
  - Add MPI_Bcast for `nnz` and `nnz_per_rank`
  - Add MPI_Bcast for matrix dimensions `m` and `n`
  - Implement vector `x` allocation on non-root ranks before broadcast
  - Add MPI_Bcast for vector `x` data
- **Step 3: Scatter matrix data**
  - Add MPI_Scatter for `row_ind` array
  - Add MPI_Scatter for `col_ind` array
  - Add MPI_Scatter for `val` array
  - Ensure proper padding calculation for equal-sized chunks
- **Step 5: Reduce results**
  - Add MPI_Reduce to combine local results from all ranks to rank 0
  - Implement proper result aggregation using MPI_SUM operation
  - Ensure rank 0 allocates memory for final result before reduction

When I started this assignment I had a light idea of how MPI worked and what functions I would need to use. For the first task, this meant checking if the world rank of the task was 0 and deciding the size of work for each of the threads. If the work could not be distributed evenly by the number of MPI processes, then another space is added to buffer those that will perform the extra work.

The second task was for the data to be broadcast to all processes. The first segmentation fault that I ran into was from trying to send the x array as I had sent the prevois data.

```
MPI_Bcast(vector_x, n,
MPI_DOUBLE, 0, MPI_COMM_WORLD);
```

Looking back, this was simply because I was not allocating memory for the array that I was sending. This process allowed the vector a buffer space as it was sent to all processes that were not rank 0.

```
if (world_rank != 0) {
    vector_x =
(double*)malloc(n * sizeof(double));
}
```

For the 3rd step, I learned how to use MPI_Scatter. The scatter operation implements a one-to-all communication pattern that uniformly partitions the sparse matrix's coordinate representation across all MPI processes. Each process receives a contiguous subset of non-zero elements, and it maintains the structural alignment between row indices, column indices, and numerical values essential for COO format. It greatly aids in parallel computation while preserving the locality of data. MPI_Scatter is an effective way to evenly split the matrix so that each MPI process gets its own piece to work on.

Finally, we were tasked with implementing the reduction and summation of the matrix - vector multiplication operation. MPI has many operations like sum, max, min, and prod which computes a product. In this task, we allowed each MPI process to complete its partial result, and then with MPI_Reduce and MPI_SUM we brought the solution back together.

3 Performance Analysis

The starting code was running on one node with 8 cores. I was not able to access more than 4 nodes at the time I was running my tests, so I decided to examine 1 node, 2 nodes, and 4 nodes with 2 cores, 4 cores, and 8 cores. I ran on average 15 to 19 tests for each of these configurations, and below there is Figure 1, which is a scatter plot of the general findings from the data I collected. I also kept tables of data that I felt were relevant, like the standard deviation of the trials, as well as the computational speedup with one node and two cores as the baseline.

As I did not have as long for this project I did not have time to additionally set up open MP with the MPI processes. I feel like this level of parallelization would have given my work more effective speedups and shown more clearly advantages of MPI processes, so this can be something that I hope to explore more in the future.

4 Results and Performance

I believe that the performance seen in the speedup must result from this misalignment of parallelization and not being aware of the node's shared memory, which gave way to poor resource utilization. Since each MPI process executes serial SpMV computation despite being allocated multiple CPU cores, a scenario is created where increased core allocation paradoxically degrades performance through memory bandwidth contention without any computational benefit. This is definitely something I did not expect when I was working through the steps, but reflecting, I can acknowledge that a second pass with OpenMP would really enhance the overall performance.

Because my configuration did not match the fundamental principle of parallel efficiency, where computational resources must be matched with appropriate parallelization granularity, I would say this was a learning experience of what is really happening with MPI processes. This investigation highlighted a fundamental insight for high-performance computing and hope to learn more about hybrid programming approaches.

I also would like to learn how the 2 dimensional implementation can also better take advantage of inner node processes. Maybe in 2 Dimensions each MPI rank owns a larger sub-domain, and the work within that sub-domain is then parallelized using threads. It also seems that combining inner node parallelism (MPI) and intra node parallelism (openMP) both need to be taken advantage of to see real speed ups.

Some final thoughts were that I found it interesting that the lower number of cores and nodes, the more varied the results for a naive COO SpMV became. I also found it interesting how the number of cores per node had a greater negative impact than the number of nodes. This can be seen when increasing the cores per node this caused computation time to spike (by up to 13 times from 2 to 8) but just increasing the nodes with fixed cores showed minimal change. This probably is because I was hitting a memory bandwidth bottleneck.

Since SpMV is memory-bound, adding cores on the same node made it more difficult for the limited memory bandwidth and it worsened cache access, especially with COO format which may not always be contiguous. I think that node count mattered less because the communication scaled well and the network wasn't the limiting factor except for the fact that I could not get more than 4 nodes.

I think optimal future strategies could be tandem using open MP, as well as using more nodes with fewer cores. I would also be interested to look at the 2D implementation in the future. MPI is an interesting frame of parallelization and I am glad we were able to have some time with this material.

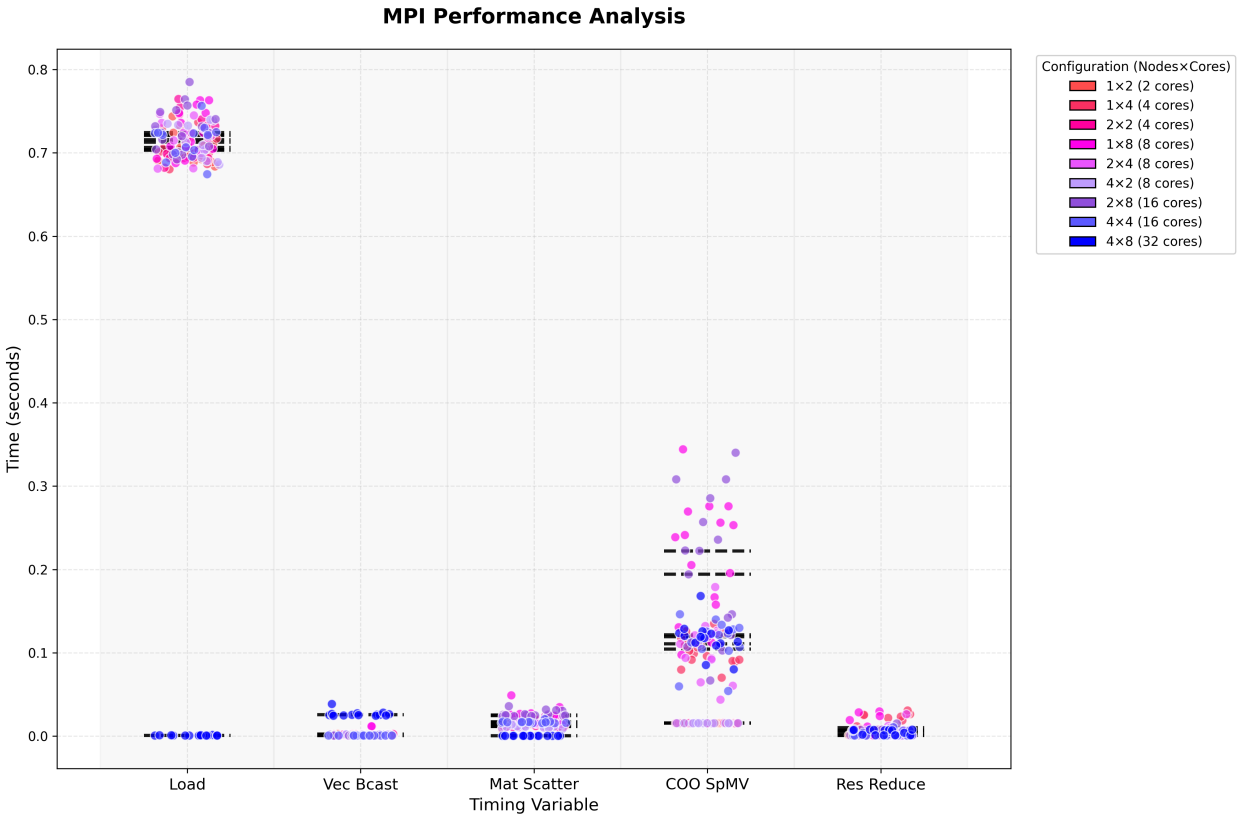

Figure 1: general timing distribution

```
============================================================
STATISTICS SUMMARY (Mean ± Std Dev in seconds):
============================================================
Config          load     vec_bcast   mat_scatter      coo_spmv    res_reduce
   1×2 0.705 ± 0.017 0.002 ± 0.000 0.013 ± 0.002 0.016 ± 0.000 0.001 ± 0.000
   1×4 0.707 ± 0.016 0.001 ± 0.000 0.018 ± 0.003 0.104 ± 0.017 0.014 ± 0.011
   2×2 0.716 ± 0.026 0.002 ± 0.000 0.013 ± 0.003 0.016 ± 0.000 0.001 ± 0.000
   1×8 0.723 ± 0.026 0.002 ± 0.004 0.027 ± 0.006 0.209 ± 0.072 0.010 ± 0.010
   2×4 0.717 ± 0.020 0.001 ± 0.000 0.016 ± 0.001 0.106 ± 0.033 0.006 ± 0.007
   4×2 0.714 ± 0.020 0.002 ± 0.000 0.012 ± 0.002 0.016 ± 0.000 0.001 ± 0.000
   2×8 0.729 ± 0.026 0.001 ± 0.000 0.027 ± 0.004 0.193 ± 0.086 0.005 ± 0.004
   4×4 0.715 ± 0.020 0.001 ± 0.000 0.017 ± 0.001 0.114 ± 0.026 0.003 ± 0.003
   4×8 0.001 ± 0.000 0.027 ± 0.003 0.000 ± 0.000 0.118 ± 0.020 0.005 ± 0.003

======================================================================
COMPUTATION SPEEDUP (COO SpMV):
======================================================================
Baseline: 1×2 = 0.0156s

1×4 (4 cores): 0.1037s  | Speedup: 0.15x | Efficiency: 3.8%
2×2 (4 cores): 0.0157s  | Speedup: 1.00x | Efficiency: 25.0%
1×8 (8 cores): 0.2091s  | Speedup: 0.07x | Efficiency: 0.9%
```

```
2×4 (8 cores): 0.1060s  | Speedup: 0.15x | Efficiency: 1.8%
4×2 (8 cores): 0.0157s  | Speedup: 1.00x | Efficiency: 12.5%
2×8 (16 cores): 0.1934s | Speedup: 0.08x | Efficiency: 0.5%
4×4 (16 cores): 0.1139s | Speedup: 0.14x | Efficiency: 0.9%
4×8 (32 cores): 0.1179s | Speedup: 0.13x | Efficiency: 0.4%
```